

Troisième phase : parallélisation des calculs dans des threads, stratégie 2

Travail en binôme : JABER ELHAM ET ELCHAMAA MOHAMAD IFA2 L3 INFORMATIQUE 2025-2026

c'est quoi overhead ?

L'overhead c'est le temps perdu à gérer les threads plutôt qu'à faire le vrai calcul.

1. Introduction

La phase 3 du projet consiste à implémenter une deuxième version parallèle du crible d'Ératosthène, basée sur une file de travaux et des threads workers. Contrairement à la phase 2 où les threads étaient créés et détruits à chaque nombre premier traité, la phase 3 maintient un pool fixe de T workers créés une seule fois au démarrage du programme.

L'objectif est de réduire l'overhead lié à la gestion des threads, tout en maintenant le parallélisme des calculs.

2. Stratégie de parallélisation

2.1 Principe de la file de travaux :

La file de travaux est un tableau circulaire de T cases fourni par l'enseignant. Chaque case contient la valeur p à traiter et un sémaphore sem_t pret qui indique si un travail est disponible. La file est gérée par fenêtre glissante : un travail reste dans la file même terminé tant que tous les travaux précédents ne sont pas finis.

2.2 Fonctionnement:

Le thread principal appelle file_travaux_ajouter() pour déposer un nombre premier p dans la file — cette fonction est bloquante si la file est pleine. Chaque worker appelle file_travaux_lire() qui bloque sur sem_wait() jusqu'à recevoir un travail. Une fois les multiples barrés, il appelle file_travaux_retirer() pour libérer sa case. Le signal d'arrêt est la valeur 0 envoyée à chaque worker.

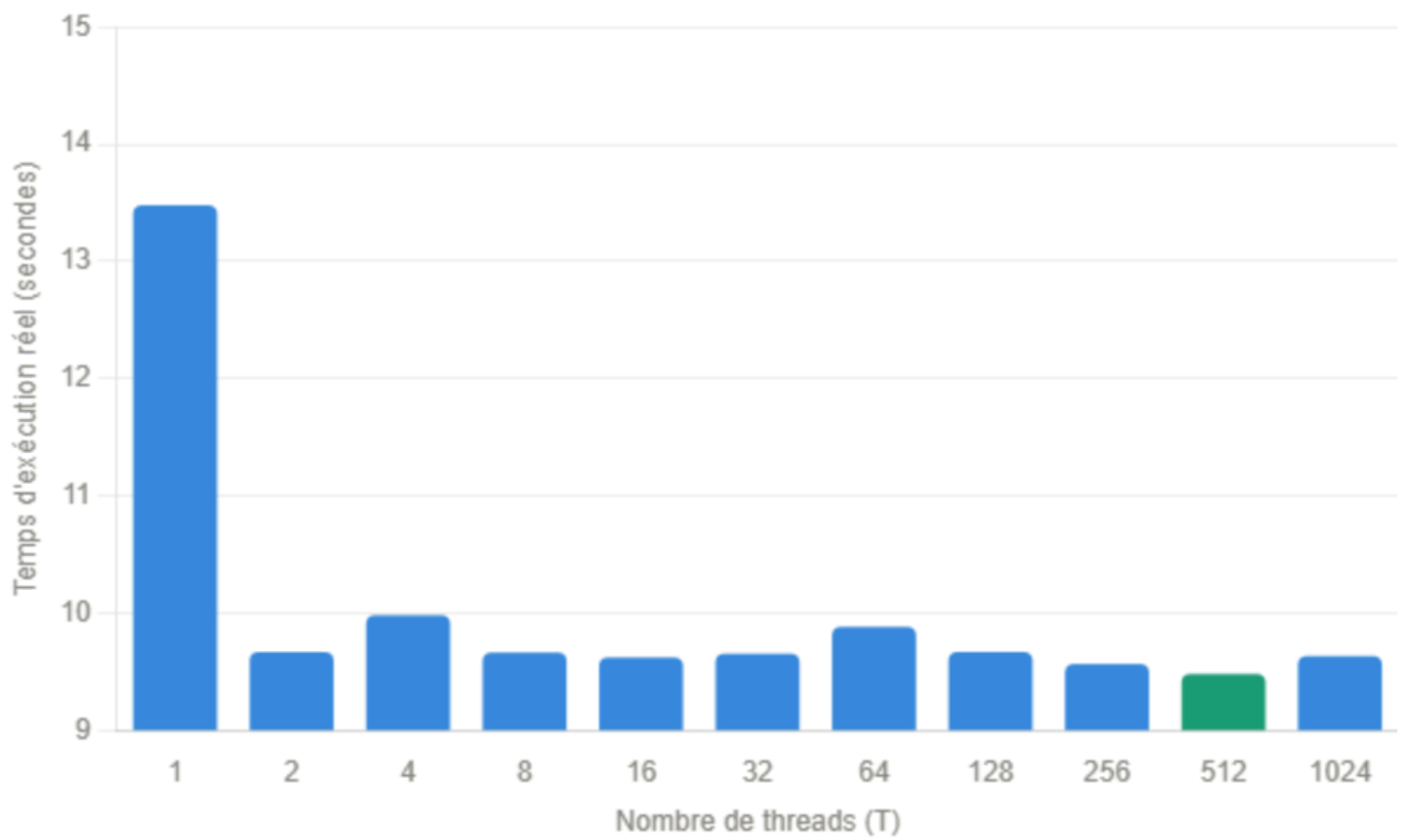
2.3 Différence avec la Phase 2

En phase 2, les threads étaient créés (pthread_create) et détruits (pthread_join) à chaque lot de nombres premiers. Ce mécanisme génère un overhead important, surtout pour les petites valeurs de T. En phase 3, les T threads sont créés une seule fois et restent actifs pendant toute la durée du programme. La synchronisation est assurée par des sémaphores , ce qui est plus efficace que de recréer des threads.

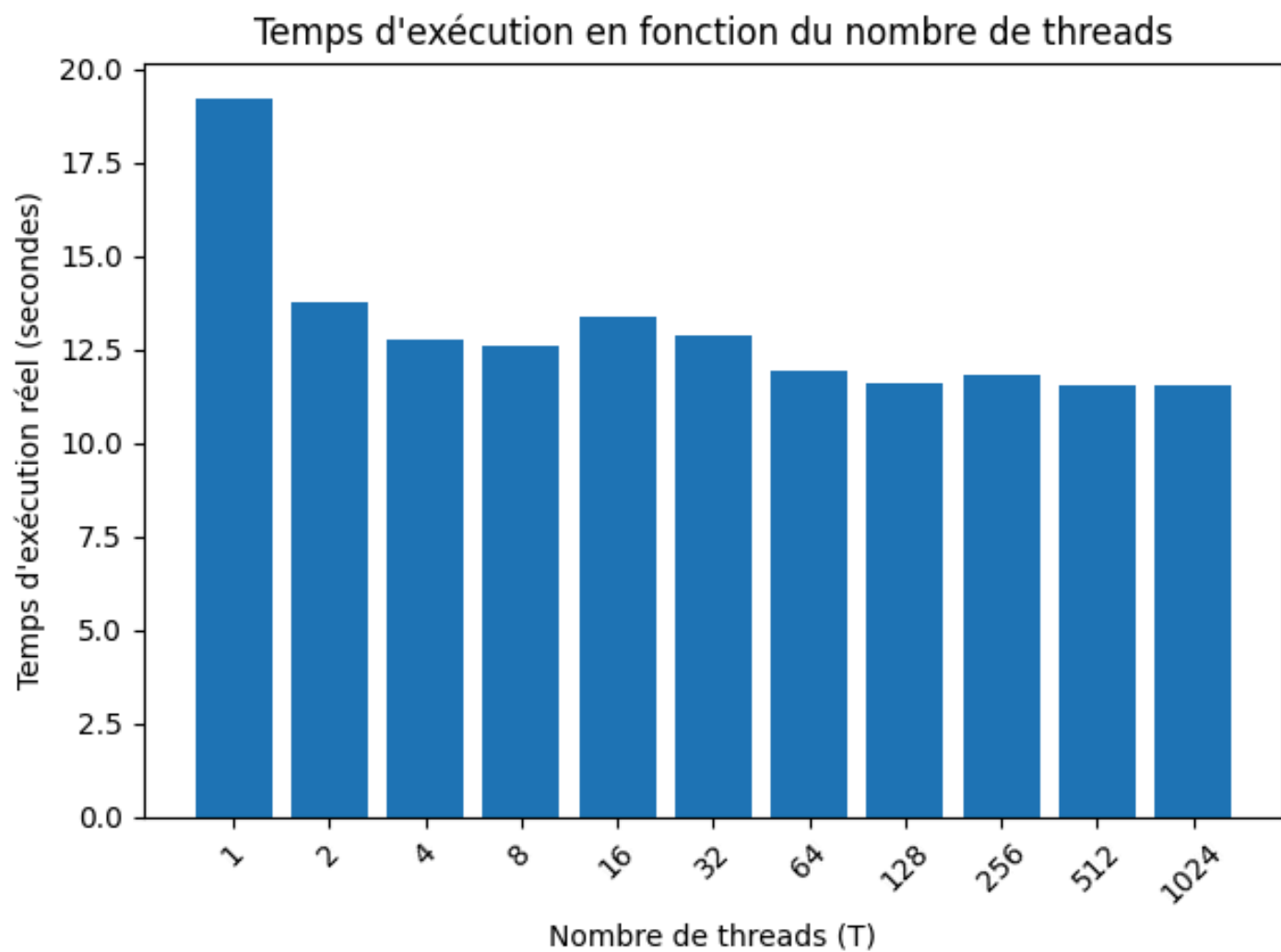
Résultats obtenus de la phase 3

N	T	CPU_TIME	WALL_TIME
1000000000	1	13.479000	13.478578
1000000000	2	9.680000	9.665866
1000000000	4	9.980000	9.979976
1000000000	8	9.663000	9.662142
1000000000	16	9.619000	9.620208
1000000000	32	9.653000	9.651850
1000000000	64	9.881000	9.881482
1000000000	128	9.666000	9.666648
1000000000	256	9.564000	9.564182
1000000000	512	9.478000	9.478882
1000000000	1024	9.626000	9.631769

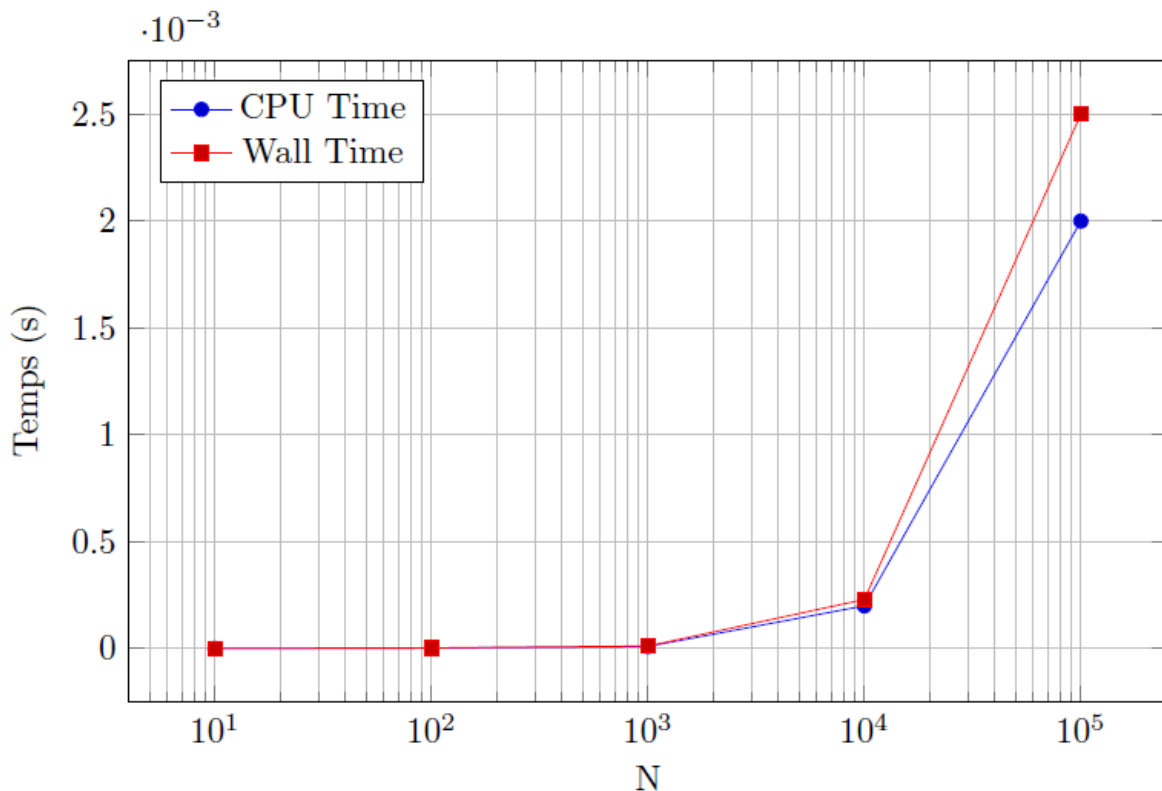
Troisième phase : parallélisation des calculs dans des threads, stratégie 2



Deuxième phase : parallélisation des calculs dans des threads, stratégie 1:



la version séquentielle de l'algorithme implémentée au début
pour la phase 1:



Comparaison des 3 approches

Phase 1 — Approche séquentielle L'algorithme s'exécute étape par étape sur un seul thread. Pour les petites valeurs de N (jusqu'à 10^6), les temps sont très faibles (moins de 5 ms). C'est l'implémentation la plus simple mais elle ne tire aucun avantage des processeurs multi-cœurs. Elle sert de référence de base.

Phase 2 — Threads parallèles Des threads sont créés et détruits à chaque nombre premier traité. Pour $N=10^9$ et $T=1$, le temps est de 19.21 s. Le gain devient visible à partir de $T=2$ (13.79 s) et se stabilise autour de 11.5 s pour $T \geq 64$. L'inconvénient principal est l'overhead lié à la création et destruction répétées de threads, ce qui pénalise les petites valeurs de T .

Phase 3 — Workers avec file de travaux Les T workers sont créés une seule fois et restent actifs pendant tout le traitement. La synchronisation est assurée par les sémaphores de la file de travaux fournie. Pour $T=1$, le temps est de 13.48 s — déjà meilleur que la phase 2 à $T=1$ car l'overhead de création de threads est absent. Le meilleur temps est atteint à $T=512$ avec 9.48 s, soit un gain de 30% par rapport au séquentiel workers. La file de travaux avec fenêtre glissante permet une meilleure gestion des cases et une libération plus efficace, ce qui explique le gain plus important qu'en phase 2.

Conclusion

Les trois approches confirment la loi d'Amdahl : le gain est limité par la partie séquentielle de l'algorithme (la recherche du prochain premier). Au-delà d'un certain nombre de threads, ajouter des workers supplémentaires n'apporte plus de gain significatif car tous les cœurs physiques sont déjà utilisés. La phase 3 reste la meilleure approche grâce à l'absence d'overhead de création de threads et à la gestion efficace de la file de travaux.

Bien Cordialement .

Elham jaber & Mohamad ELchamaa